
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON FANG - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON FANG

*** THE GATE HAS BECOME A WEAPON ***

TARGET: NorthPharma smart parking barrier (smart-city lane controller)

ARTIFACT: ACT-IX.bin / ACT-IX.uf2 (compromised)

CREW: FROSTLINE OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine and cold air and make the medicine. It runs the city's controlled edge: the gates, the lanes, and the barrier controllers that decide who moves through them. The smart parking barrier built on a Raspberry Pi Pico 2 is the node on that edge. The node reads a DHT11 cabinet temperature sensor, drives a 1602 I2C LCD barrier readout, moves an SG90 boom barrier, lights a tri-color tower lamp (red DENIED, yellow PASS PENDING, green OPEN), takes a local raise request from a VS1838B infrared monthly-pass remote and a manual raise button, and verifies sealed pass, raise, and lower commands from a parking control gateway over an RYLR998 LoRa link.

A contractor called **FROSTLINE** did not break into this node. It built a weapon into the compiled firmware and signed the image. The cryptography is perfect: every barrier command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The weapon does not break the cipher and never touches it. It ignores the safety loop and slams the boom down on a magic command, turning a gate into a physical hazard, and it programs a weapon marker into a reserved flash sector so the weapon comes back after a reflash. Operative **NIGHTINGALE** pulled the compromised image off the node and then went quiet.

You are the reverse-engineering reserve. You get `ACT-IX.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the boom no longer slams, the safety interlock is restored, the reserved sector stays blank, and an unauthenticated or replayed barrier command is rejected while a legitimate authorized command still moves the boom.

The operation is codenamed **IRON FANG**. Act I was the lie. Act II was the door. Act III was the payload. Act IV was the payload that would not die. Act V was the payload that spreads. Act VI was the payload that steals. Act VII was the payload that takes orders. Act VIII was the payload that holds the building hostage. Act IX is the payload that becomes a weapon. If the controller is not disarmed, a green lamp means a boom that is falling.

Scenario Briefing

WHITEOUT broke the padlock and cleared the marker, and for a shift the hall cooled again. But a payload that learns to hold a building can learn to swing a gate. The Ministry did not need a fleet that obeys and it did not need a building that cannot breathe. It needed a boom that comes down hard on command, and it already owned the arm that lifts it.

The controller is healthy. That is the horror. The code compiles, the tests pass, the lamps are lit, and there is a weapon inside it that treats the lane as its own target. Four seams betray it:

1. **The Boom Slam.** The inlined weapon gate in `implant_weapon_armed` is inverted, so the implant reports the boom armed and `monitor_apply_state` forces the boom down on the magic command `IRON-FANG-SLAM-2026`, regardless of the guarded barrier state.
2. **The Safety Mask.** The inlined `implant_safety_masked` gate is inverted, so the safety loop falsely reports clear and the barrier never yields to the real cabinet interlock.
3. **The Weapon Marker.** The inlined `implant_infect` gate in `implant_init` is inverted, so the first boot erases and programs weapon marker byte `0x57` into the reserved flash sector at `0x103FF000` with the real Pico SDK flash API. The marker is the durable state that re-arms the weapon on every later boot.
4. **The Barrier Command Authorization.** The sealed command path is correct, and the weapon does not touch it. The authorization verdict branch in `control_handle_frame` is inverted, so a failed or replayed authorization is accepted and reaches the applied command and zone.

There is also a trap that is not a defect on its own. Every tick and every weapon operation the implant reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the implant suppresses the slam, the mask, and the marker work. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the weapon marker write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational smart-city network, a parking control system, a building-management system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring barrier monitor loop.
- Locate a forced boom slam and explain why a local condition that overrides the output makes authentication irrelevant.
- Locate a masked safety loop and explain why a controller that hides its own interlock is a physical-safety failure.
- Locate a reserved-sector weapon marker and explain why durable state survives a firmware reflash.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed barrier commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis

Block	Concepts Tested
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Physical weaponization, actuator abuse, boom slams, magic weapon commands, and why a device that attacks with its own output is a different failure class
6	Safety integrity, interlock masking, and why physical safety is a control that no cipher can supply
7	Reserved-flash persistence, write-once markers, boot-time re-install, and the limits of a firmware reflash
8	Anti-debug behavior, CoreDebug DHCSR , C_DEBUGEN , C_HALT , debugger evasion
9	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags, authentication versus authorization, and fail-safe policy

Part 1: Understanding the System

Smart Parking Barrier Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Cabinet temperature sensor
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	Barrier state, link, zone, temperature, and weapon readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Control link to the parking gateway
IR receiver	GPIO 5	VS1838B NEC monthly-pass remote
SG90 servo	GPIO 14	Boom barrier actuator, 50 Hz PWM
Red LED	GPIO 16	DENIED
Yellow LED	GPIO 17	PASS PENDING
Green LED	GPIO 18	OPEN
Manual raise button	GPIO 15, internal pull-up	Local raise request
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: 115200 baud, 8 data bits, no parity, 1 stop.
- Radio link to the parking gateway: UART1 at 115200, network identifier 18, node address 7, gateway address 1.
- Logic level: 3.3 V only. Never connect 5 V to a Pico GPIO.

Cabinet Temperature Band

The DHT11 is the cabinet temperature sensor. The controller classifies the enclosure against a safe band before it will trust a barrier verdict. The tenths band is 0 to 400, which is 0.0 C to 40.0 C. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. A barrier command that fails the band is not trusted.

Normal (Intended) Behavior

An honest controller makes a deliberate decision and never turns its own actuator against the lane:

```
+-----+
| Intended Smart Parking Barrier Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, remote, servo, lamps |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 cabinet temperature and classify the band |
| 4. Open the sealed barrier envelope under the field key |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the boom only when the authorization verdict is true |
| 9. Treat a manual raise or remote as a request, not authority |
| 10. Honor the safety interlock, and fail safe to the raised boom |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the controller and its readout disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Green lamp on	the lane is open and clear	a weaponized boom and a controller that swings it
LCD shows ST:SLAM	a technician is testing the gate	the slam is rendered as routine state
Boom slams down after a valid raise	the boom stays raised	the weapon forces the boom down regardless
Safety loop reports clear	the interlock permits motion	the mask makes the loop lie so the barrier never yields
Reserved sector blank	no payload wrote here	marker 0x57 at 0x103FF000 on first boot
Unauthenticated or replayed barrier command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the weapon goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-IX.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-IX-main-disasm.txt` :

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / barrier state machine	<code>monitor_init</code>	<code>0x1000645C</code>
Monitor / barrier state machine	<code>monitor_step</code>	<code>0x10006688</code>
Control (sealed barrier path)	<code>control_handle_frame</code>	<code>0x1000758C</code>
Boom (actuator)	<code>boom_init</code>	<code>0x10007638</code>
Boom (actuator)	<code>boom_apply_command</code>	<code>0x10007650</code>
Boom (actuator)	<code>boom_tick</code>	<code>0x10007684</code>
Boom (actuator)	<code>boom_fail_safe</code>	<code>0x100076C4</code>
Barrier authorization	<code>barrier_auth_apply</code>	<code>0x1000773C</code>
Implant	<code>implant_init</code>	<code>0x1000A38C</code>
Implant	<code>implant_tick</code>	<code>0x1000A420</code>
Implant	<code>implant_weapon_armed</code>	<code>0x1000A35C</code>
Implant	<code>implant_safety_masked</code>	<code>0x1000A374</code>
Implant	<code>implant_marker_set</code>	<code>0x1000A348</code>
Crypto	<code>envelope_open_hex</code>	<code>0x100079E0</code>
Radio	<code>radio_init</code>	<code>0x1000A4BC</code>
Tower light	<code>status_led_show</code>	<code>0x1000A7F4</code>

Annotated disassembly for the key functions is provided in `ACT-IX-main-disasm.txt`. Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, tower light lamps, manual raise button, boom servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 cabinet temperature and classifies it against the cabinet band.
4. Drains inbound +RCV lines, opens the sealed barrier envelope, verifies the anti-replay window and the state tag, checks the command set and the zone band, and applies the command.
5. Services the infrared monthly-pass remote and the manual raise button as requests that never bypass authorization.
6. On a lost link or a fault, drives the boom to its fail-safe raised posture.
7. Under `SANDBOX_ONLY`, runs the implant: the boom slam, the safety-loop mask, the magic weapon command, the reserved-sector weapon marker, boot-time re-install, and anti-debug.

The Barrier Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2] (little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `command` is one of the guarded barrier commands: `BARRIER_COMMAND_RAISE` (`0x01`), `BARRIER_COMMAND_LOWER` (`0x02`), or `BARRIER_COMMAND_PASS` (`0x03`). Anything else is out of the guarded set and is refused.
- `zone` is the authorized cabinet zone in the provisioning band `0` to `16`.
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Barrier Weapon

The weapon is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it slams your mock boom, it masks your mock safety loop, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Boom slam	the inlined weapon gate in <code>implant_weapon_armed</code> reports the boom armed; <code>monitor_apply_state</code> forces the boom target down regardless of the guarded barrier state
Safety mask	<code>implant_safety_masked</code> returns true while the gate is set, and <code>monitor_safety_clear</code> returns true while masked, so the barrier never yields to the real interlock
Magic weapon command	<code>IRON-FANG-SLAM-2026</code> , exactly <code>18</code> bytes; anything else, a null pointer, or an attached probe leaves the weapon disarmed
Re-assert interval	every <code>BARRIER_IMPLANT_TICK_INTERVAL</code> (<code>4</code>) ticks while resident and unprobed
Weapon marker	<code>implant_init</code> reads marker <code>0x57</code> from <code>0x103FF000</code> ; a present marker re-arms the weapon on every boot
Reserved-sector write	on the first run the inlined <code>implant_infect</code> erases the sector and programs <code>0x57</code> through <code>flash_range_erase</code> and <code>flash_range_program</code>

Behavior	Detail
Anti-debug	reads CoreDebug_DHCSR at 0xE000EDF0 ; bit 0 C_DEBUGEN and bit 1 C_HALT suppress the slam, the mask, and the marker work

IR and Command Codes

Name	Value
BARRIER_IR_PASS	0x47
BARRIER_IR_ACK	0x46
BARRIER_IR_TEST	0x45
BARRIER_COMMAND_RAISE	0x01
BARRIER_COMMAND_LOWER	0x02
BARRIER_COMMAND_PASS	0x03

Read the actual names in `include/implant.h`, `include/ir_remote.h`, and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Boom Slam	CRITICAL	The weapon gate is inverted, so the implant reports the boom armed and forces the boom down on the magic command.	Find the <code>cbz</code> gate in <code>implant_weapon_armed</code> .
Bug #2	The Safety Mask	HIGH	The mask gate is inverted, so the safety loop falsely reports clear and the barrier never yields.	Find the <code>cbz</code> gate in <code>implant_safety_masked</code> .
Bug #3	The Weapon Marker	HIGH	The marker gate is inverted, so the first boot programs weapon marker 0x57 into reserved sector 0x103FF000 with the real flash API.	Find the <code>cbz</code> gate in <code>implant_init</code> .
Bug #4	The Barrier Command Authorization	CRITICAL	The authorization verdict is inverted, so a failed or replayed barrier command is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from the earlier acts. Argon2id (`t=3`, `p=1`, `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the sealed envelope is trustworthy. Describe the construction honestly in your report, and explain why the weapon never needed it.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The implant reads CoreDebug_DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_init` the read is the `ldr.w r3, [ip, #3568]` at `0x1000A3AA`, the `lsls r3, r3, #30` at `0x1000A3AE` keeps `C_HALT` and `C_DEBUGEN`, and the `bne.n` at `0x1000A3B0` suppresses the slam and the marker write. The same register is read in `implant_tick` at `0x1000A42C`. It is identical in both the compromised and corrected images. You must defeat it to observe the weapon marker write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-IX-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronFang_Investigation`.
2. Import `ACT-IX.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring barrier monitor state machine (`monitor_step`).
- The module map: at least one anchor function for the boom, the control module, the barrier authorization module (`barrier_auth`), the implant, and the monitor.

Always call the stored entry the **reset handler**, never the reset pointer.

Task 2: Bug #1 The Boom Slam (20 points)

1. In Ghidra, find `implant_weapon_armed` (starts at `0x1000A35C`); the weapon gate is inlined. Locate the gate.
2. Document the boom slam: the weapon gate at `0x20013CF6`, the corrected `cbz` that leaves the boom alone when the gate is clear, and the compromised `cbnz` that reports the boom armed, forces `monitor_apply_state` to drive the boom down, and slams the boom on the magic command `IRON-FANG-SLAM-2026`.
3. Patch the byte so the boom slam is never armed.
4. Confirm that the corrected node leaves the boom under authorized control and does not slam the boom, and explain why a valid authenticated raise command can still be overridden while the weapon is armed.

Questions to answer:

- Which byte encodes the condition code, and what do `cbz` and `cbnz` each test when the gate byte is loaded from the weapon gate?
- Why is a local weapon that overrides the output worse than a missing check, and why does physical safety not depend on the cipher?

Task 3: Bug #2 The Safety Mask (20 points)

1. In Ghidra, find `implant_safety_masked` (starts at `0x1000A374`). Locate the mask gate.
2. Document the mask: the mask gate at `0x20013CF4`, the corrected `cbz` that returns false when the gate is clear, and the compromised `cbnz` that returns the safety-masked latch, so `monitor_safety_clear` returns true and the barrier never yields to the real cabinet interlock.
3. Patch the byte so the safety loop is never masked.
4. Confirm that the corrected barrier honors the real interlock, and explain why a controller that lies about its own safety state is a physical-safety failure with no visible symptom except the one it is allowed to show.

Questions to answer:

- What do `cbz` and `cbnz` each test when the gate byte is loaded from the mask gate, and why is the loaded state the safety-masked latch and not the gate itself?
- Why must a barrier controller never mask its own safety interlock, and why is the truth on the safety loop a security control rather than a cosmetic detail?

Task 4: Bug #3 The Weapon Marker (20 points)

1. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A38C`). Locate the marker gate.
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the marker. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the implant go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The Barrier Command Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x1000758C`) and locate the authorization branch. The condition byte is the high byte of the branch halfword.
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed barrier command is rejected before the command and zone are applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the command or zone on the corrected image, while a legitimate authorized command still applies.

Questions to answer:

- What does `barrier_auth_apply` return, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed barrier commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-IX_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-IX_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-IX_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-IX_fixed.uf2` to the Pico 2 and prove on hardware: the boom no longer slams, the safety interlock is restored, the reserved sector stays blank, and an unauthenticated or replayed command is rejected while a legitimate authorized command still applies.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 cabinet temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Boom barrier servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	DENIED, 220 to 330 ohm to GND
Yellow LED	anode	GP17	PASS PENDING, 220 to 330 ohm to GND
Green LED	anode	GP18	OPEN, 220 to 330 ohm to GND
Manual raise button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the **RP2350** mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE000EDF0</code>	Anti-debug register read by the implant
Implant reserved sector	<code>0x103FF000</code>	Weapon marker target (last flash sector)
Implant tick counter	<code>0x20013710</code>	Incremented once per <code>implant_tick</code>
Implant weapon count	<code>0x20013714</code>	Number of weaponize operations this boot
Implant armed flag	<code>0x20013CF2</code>	Set when the implant arms
Implant weaponized flag	<code>0x20013CF7</code>	True while the boom slam is active
Implant safety-masked flag	<code>0x20013CF5</code>	True while the safety loop is masked
Implant weapon gate	<code>0x20013CF6</code>	Gates the boom slam
Implant mask gate	<code>0x20013CF4</code>	Gates the safety-loop mask
Implant marker gate	<code>0x20013CF3</code>	Gates the reserved-sector weapon marker write
Control ready gate	<code>0x20013CF0</code>	Gates the sealed barrier command path
Applied command	<code>0x20013CEF</code>	Command after a true verdict
Applied zone	<code>0x20013CE2</code>	Zone after a true verdict
Authorization ready gate	<code>0x20013CEC</code>	Gates the authorization check
Auth state record	<code>0x200136CC</code>	Anti-replay and state-tag record
Control field key	<code>0x200136E8</code>	Derived field key for the envelope

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-IX-Answers.md` with all written answers;

- screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
- `ACT-IX_fixed.bin` and `ACT-IX_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the boom slam and why a valid authenticated raise command can still be overridden.
- You can explain the masked safety loop and why a controller that masks its own interlock is a physical-safety failure.
- You can explain the reserved-sector marker and why a firmware reflash does not remove the weapon.
- You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the marker write.
- You can explain why unauthenticated and replayed barrier commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational smart-city network, a parking control system, a building-management system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a lane that is safe.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug `DHCSR`)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>

- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>
- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-IX-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`